



Boosting File Systems Elegantly: A Transparent NVM Write-ahead Log for Disk File Systems

Guoyu Wang, Xilong Che, Haoyang Wei, Shuo Chen, Puyi He,
and Juncheng Hu, *Jilin University*

<https://www.usenix.org/conference/fast25/presentation/wang>

This paper is included in the Proceedings of the
23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings
of the 23rd USENIX Conference on
File and Storage Technologies
is sponsored by



Boosting File Systems Elegantly: A Transparent NVM Write-ahead Log for Disk File Systems

Guoyu Wang, Xilong Che, Haoyang Wei, Shuo Chen, Puyi He, and Juncheng Hu

Jilin University

Abstract

We propose NVLog, an NVM-based write-ahead log for disk file systems, designed to transparently harness the high performance of NVM within the legacy storage stack. NVLog provides on-demand byte-granularity sync absorption, reserving the fast DRAM path for asynchronous operations, meanwhile occupying NVM space only temporarily. To accomplish this, we designed a highly efficient log structure, developed mechanisms to address heterogeneous crash consistency, optimized for small writes, and implemented robust crash recovery and garbage collection methods. Compared to previous solutions, NVLog is lighter, more stable, and delivers higher performance, all while leveraging the mature kernel software stack and avoiding data migration overhead. Experimental results demonstrate that NVLog can accelerate disk file systems by up to 15.09x and outperform NOVA and SPFS in various scenarios by up to 3.72x and 324.11x, respectively.

1 Introduction

Non-volatile memory (NVM), with its persistent and byte-addressable characteristics, is emerging as a new tier in the memory hierarchy, poised to accelerate lower storage devices while ensuring data consistency. However, the unique properties of NVM, combined with traditional DRAM and disk, create a complex heterogeneous system. The complexity presents both opportunities and challenges for software design.

To leverage NVM, various NVM-specific file systems have been proposed. Among them, NVM-specialized kernel file systems [1, 16, 40] are the most popular due to their compatibility with the existing kernel software stack, allowing existing applications to transition to NVM file systems with minimal cost. Unlike traditional block-based file systems, these file systems prioritize the byte-addressable and persistent nature of NVM and achieve higher performance. However, the significantly lower capacity of NVM compared to block storage devices limits the widespread adoption of NVM-specialized file systems. Additionally, many well-known and widely accepted NVM file systems, such as NOVA, utilize direct access

(DAX) to reduce the need for copying data from DRAM to NVM. As we will show in Figure 1, the relatively slower speed of NVM can make these file systems less efficient than page cache in many scenarios.

There are also cross-media file systems [21, 43] designed to exploit the heterogeneous performance and capacity characteristics of different storage tiers, typically including DRAM, NVM, and block storage. However, due to their complex designs, these systems tend to be less mature and robust than widely used, time-tested file systems like Ext-4. Furthermore, the redesigned architecture for each media tier renders them incompatible with current file systems used by online systems, leading to high data migration costs for potential users.

The latest trend in utilizing NVM focuses on accelerating existing block device file systems. SPFS [37] and P2CACHE [26] are two examples of NVM file systems layered on top of traditional disk file systems, aiming to enhance disk file systems by leveraging NVM. Generally, these approaches use NVM to speed up the slow persistence processes of disks. However, the speed improvements often come with certain side effects. For instance, SPFS introduces a second indexing overhead due to its overlay design and suffers from reduced re-access speed once data is absorbed by its NVM component. P2CACHE employs NVM to absorb not only synchronous writes but also asynchronous ones to ensure strong consistency, which unnecessarily slows down asynchronous writes. These drawbacks arise from their multi-tiered, stacked design, which fails to place NVM in its optimal position.

This poses a complex problem: **how to fully utilize the attractive characteristics of NVM while maintaining the advantages of the traditional software stack, ensuring compatibility and transparency for current user programs, and avoiding performance degradation and migration costs?** To address this, we propose NVLog, a transparent NVM write-ahead log designed to accelerate the performance of current disk file systems. Like SPFS and P2CACHE, NVLog aims to optimize the performance of existing mature disk file systems with NVM transparently. The unique contribution of NVLog lies in two aspects: (1) NVLog fully

preserves the advantages of the DRAM page cache, ensuring that the use of NVM does not cause performance degradation in any use case; (2) NVLog is implemented as a log lies aside current VFS page cache, rather than an overlay file system, making it more efficient and compatible compared to previous work.

Though using faster media to accelerate slower ones has been a common practice in computer systems, things become different when it comes to NVM. The heterogeneity of DRAM, NVM, and disks [38] presents significant challenges. First, these diverse devices differ in access granularity, speed, capacity, and price. Achieving optimal performance across all use cases requires leveraging the strengths and mitigating the weaknesses of each device, a task proven to be difficult by previous work [26, 37]. Second, to fully exploit the performance of NVM, a meticulous design and implementation are necessary to prevent the software stack from becoming the primary bottleneck [24]. Finally, ensuring consistency across heterogeneous devices can be challenging, especially when writes are performed on different devices with varying timing and granularity.

NVLog records, and only records, synchronous writes into its NVM log structure, while preserving the DRAM cache to ensure that the performance of other normal operations remain unaffected. To maintain eventual consistency on the disk file system, the sequence of NVM syncs and disk write-backs is strictly defined. Additionally, NVLog optimizes the trigger mechanism of synchronous operations to avoid write amplification caused by scattered small writes. Furthermore, unlike previous work, NVLog’s efficient write-ahead log design only requires a small portion of the NVM space, so the remaining free space can be utilized to support tiered caching or to serve applications that wish to use NVM directly.

The remainder of the paper is organized as follows. Section 2 describes the characteristics of NVM and reviews related work on NVM. Section 3 presents our insights on heterogeneous storage system and the design principles of NVLog. Section 4 provides the detailed design of NVLog. Section 5 discusses the implementation details. Section 6 evaluates the performance of NVLog. Finally, Section 7 concludes the paper.

2 Background

2.1 Non-volatile Memory

Non-volatile memory (NVM) is a type of byte-addressable memory that can persist data. According to the JEDEC specification [5], NVM can be categorized into three types: NVDIMM-F, which uses flash storage on a DIMM; NVDIMM-N, which combines flash and DRAM on the same module, typically with a backup power source; and NVDIMM-P, which is inherently persistent as the computer’s main memory, including technologies such as PCM, RRAM, and STT-

RAM [4, 18, 22, 36]. Intel Optane [4] has been the most popular NVM (NVDIMM-P) technology in recent years. Although Intel has discontinued Optane, alternative technologies have emerged in the market [8, 11], and we anticipate various substitutes in the future. In our work, NVM mainly refers to *NVDIMM-P*, which is the most widely used technology, but our methods can also be applied to NVDIMM-N modules.

Typically, NVM offers intermediate performance, capacity, and price characteristics between DRAM and SSDs [16, 19, 38, 39], which presents both opportunities and challenges for interested software [38, 41]. Efforts to utilize NVM can be broadly categorized into three approaches.

The most efficient method is to expose NVM directly to user-space programs, allowing them to manipulate NVM with minimal software stack latency [12, 14, 15, 20, 28, 32–34, 44]. This approach is ideal for new programs designed to fully leverage NVM. *However, for existing programs, particularly large and complex ones, migrating to new hardware with a new access pattern requires substantial effort, which can sometimes be impractical.*

A simpler concept is to use NVM as a slow second-tier memory for caching relatively cold data [7, 17, 29–31, 35]. This method takes advantage of NVM’s price and capacity benefits, *but does not leverage its persistence characteristics.*

A more balanced approach is to incorporate NVM file systems within the kernel. Since applications typically interact with persistent storage devices through kernel file systems, integrating NVM file systems can be seamlessly applied to existing applications.

2.2 File Systems for NVM

Various file systems have been proposed to exploit the performance of NVM devices. Previous work can be broadly categorized into two types: NVM-specialized file systems and cross-media file systems.

NVM-specialized file systems: Unlike traditional file systems designed for slower block storage devices like SSDs and HDDs, NVM-specialized file systems are tailored to leverage the unique characteristics of NVM. Direct Access (DAX) [1] was an early attempt to bypass the DRAM page cache in traditional file systems. By using DAX, block file systems can operate directly on NVM without additional DRAM copies. DAX can be applied to various block file systems, such as Ext-4 and XFS. However, as a patch on block file systems, DAX does not fully exploit the byte-addressable nature of NVM. It still incurs unnecessary software overhead and lacks proper consistency guarantees. Despite these limitations, the concept of DAX has influenced subsequent work.

For some later efforts, NOVA [40] is a dedicated file system for NVM, designed to leverage its fast and persistent characteristics. NOVA introduces a specially designed log structure on NVM to provide strong consistency and high performance. However, as shown in Figure 1, NOVA sometimes

performs worse than traditional file systems with DRAM page cache due to the relatively lower speed of NVM. Additionally, its copy-on-write (CoW) design does not fully capitalize on NVM's byte-addressable access pattern. Besides, high-performance user-space (or user-kernel hybrid) file systems [12, 15, 27, 33, 44] have also been proposed to reduce latency caused by kernel traps but are not widely adopted due to compatibility issues with existing applications. *These NVM-specialized file systems focus on persisting data on NVM but do not address the intermediate speed and capacity of NVM compared to DRAM and SSDs.*

Cross-media file systems: Cross-media file systems introduce monolithic file systems with a holistic view over multiple heterogeneous storage tiers. These file systems are typically deployed across DRAM, NVM, SSD, and HDD. Strata [21] uses a user-space LibFS with per-process update logs to accelerate single-process access. Data is then digested into a shared area protected by the kernel and strategically placed into different media. Ziggurat [43] aims to accelerate slower disk storage with NVM, providing a file system with the speed of NVM and the capacity of larger disks. By predicting user write operations, Ziggurat dynamically directs operations to the appropriate storage tier. These cross-media file systems offer a comprehensive solution for balancing speed, capacity, and price by utilizing DRAM, NVM, and disk. *However, the monolithic design of these systems makes them difficult to tailor and deploy. Additionally, due to their complexity, a significant amount of time may be required for verification and modification before they reach maturity.*

2.3 Enhancing Disk File Systems with NVM

A novel method to utilize NVM is to enhance existing disk file systems. The earliest attempts in this area involved transferring the journals of existing disk file systems to NVM. Since file systems with journaling incur 2.7 times higher write traffic than those without journals [23], some works [6, 42] utilize NVM to absorb the commit and checkpoint operations of the journal, thereby reducing the write traffic to the disk. *However, such approaches can only accelerate the journaling phase, and data still needs to be written to disk. Therefore, in synchronous scenarios, the overhead of direct disk I/O remains unavoidable.*

Another approach is to use NVM to transparently accelerate the synchronous writes of existing disk file systems, which are inherently slow due to their immediate persistence requirements and the slow speed of disk I/O. Although synchronous writes occur less frequently than normal read/write operations, they often become the main bottleneck of disk file systems, especially under workloads with strict correctness requirements, such as databases.

SPFS [37] is a stackable file system that lies on top of a traditional disk file system. Small synchronous write operations are directed to the overlaid NVM layer to eliminate

I/O costs associated with slower disks, while other writes are handled by the lower file system. Since the page cache is maintained in the lower layer, non-synchronous read and write operations can still benefit from the speed of DRAM. SPFS takes into account the different performance characteristics of DRAM and NVM. However, SPFS relies on predictions to absorb synchronous writes, with these predictions based on the access patterns of previous synchronous writes. Before a successful prediction is made, the system still suffers from the low performance of synchronous I/O. This is an obvious problem when synchronous operations occur without a regular pattern. Additionally, once data is directed to the upper NVM layer, subsequent read operations also have to be performed on NVM, leading to unnecessarily high latency. Moreover, the two-layer design of SPFS introduces a double-indexing overhead, which lowers the overall performance of the system.

P2CACHE [26] is another overlay file system. There are two key differences between P2CACHE and SPFS. First, P2CACHE positions the DRAM cache to the same level as the NVM layer. Any data is written to both NVM and DRAM simultaneously, allowing P2CACHE to serve subsequent read operations from the faster DRAM. Second, P2CACHE absorbs not only synchronous but also normal write operations to its NVM layer to provide strong consistency. Although P2CACHE claims to benefit from overlapping writes between NVM and DRAM, for normal writes, writing to NVM is still slower than writing to DRAM. Furthermore, as an experimental work, P2CACHE fails to support several necessary functions, such as page status management, page locking, and reclamation support, which are indispensable for a real operating system. Additionally, implementing page cache as non-volatile also requires changes from applications, as they previously assumed that the page cache was volatile.

Overall, these overlay file systems fail to put NVM into the proper position of the memory hierarchy and do not fully exploit its advantages, as we will illustrate in Section 3, resulting in limited acceleration.

3 Motivation

In current operating systems, storage devices are typically managed by file systems. We evaluated the basic performance of a traditional disk file system, Ext-4, with and without page cache, and compared it with file systems on NVM (NOVA, Ext-4, and Ext-4-DAX). The testbed of this evaluation is the same as that of Section 6. Figure 1 shows that operations performed on the DRAM page cache can always achieve higher performance than those on NVM. *The main limitation of current disk file systems lies in sync writes and cache-missing operations.* We also performed a breakdown of the access latency. The results show that the slowdown of these cache-absent operations is primarily due to synchronous I/O and additional software stack overheads. For cache-missing reads

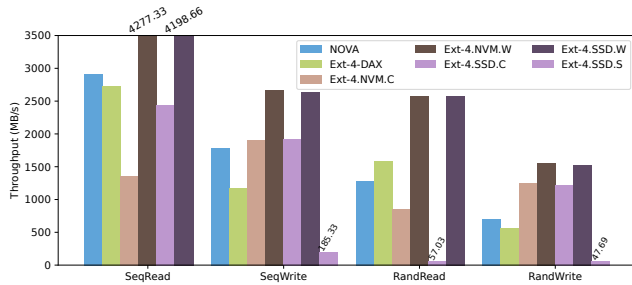


Figure 1: The throughput on different file systems and different storage devices, tested with FIO. C and W suffixes indicate that the page cache is cold (cache miss) or warm (cache hit). S means sync writes. Reads are not affected by sync.

and sync writes, the direct I/O requirement is the main contributor (over 90%) to the slowdown. For cache-missing writes, although data can be written to the DRAM cache without I/O, memory allocation and index building for the absent pages become the primary factors (70%) of performance degradation.

The observation is simple and intuitive: performance degradation occurs only when a faster storage layer, in this case, the DRAM cache, is missing. Therefore, we believe that preserving an efficient, high-speed DRAM cache, while accelerating cache-missing paths—sync writes and cache-absent operations—with NVM, presents a promising approach to further enhancing the current storage software stack.

Sync writes are common in strong-consistency workloads, such as databases, which form the backbone of many Internet applications. In such scenarios, we aim to provide acceleration similar to that of NVM file systems—by absorbing foreground sync writes to faster NVM instead of writing to disk—thereby optimizing throughput and latency. However, in cases where sync writes are infrequent, NVM file systems are less competitive than disk-based file systems because they fail to efficiently utilize DRAM. In these situations, we strive to maintain the fast DRAM path while also accelerating slow sync writes, thus providing further performance improvements rather than degradation.

For cache-missing async operations, extending the size of the page cache can be beneficial. Methods [7, 17, 29–31, 35] have been proposed to extend the DRAM cache with large and cheap NVM, known as tiered memory; however, they usually do not integrate well with NVM file systems, as these file systems often require a large NVM space. We aim to further support tiered memory by reducing our NVM usage.

In summary, the goal of this work is to use NVM to accelerate the current storage system efficiently and transparently, without any performance downgrade or data/code migration, while requiring only minimal NVM persistent space, allowing the remaining space to be used for other

purposes such as extending the page cache. We hence introduce NVLog, an NVM-based write-ahead log integrated within the VFS page cache, to efficiently accelerate the slow sync writes of current disk file systems while maintaining transparency to user applications and compatibility with the system storage software stack.

NVLog is based on two insights that were ignored by previous disk file system accelerators (Section 2.3):

- I1:** The DRAM cache is sufficient and efficient to serve applications. Therefore, when persisting synchronized data to NVM, the focus should be on the efficiency of *recording*, rather than *data retrieval*.
- I2:** Establishing a well-defined write timing between NVM and disk is crucial for ensuring crash consistency while minimizing the amount of data written to NVM.

Due to neglecting **I1**, both P2CACHE [26] and SPFS [37] have to create an index for data on NVM for subsequent reads, and have difficulty reducing the space usage on NVM. Due to neglecting **I2**, these systems are forced to also redirect async writes to NVM when absorbing sync writes, in order to avoid inconsistencies between the data from sync writes (to NVM) and async writes (to disk).

To respond to the insights and achieve our goals, NVLog is designed and implemented with four principles:

- P1: Transparency to both upper applications and lower file systems.** NVLog should function as a transparent file system accelerator, requiring no changes to applications to benefit from the acceleration. Additionally, NVLog should not necessitate any modifications to the underlying file systems, allowing all time-tested disk file systems to be accelerated without additional cost. This downward transparency also means that expensive data migration is not required for users.
- P2: No consistency change to the current I/O stack.** NVLog should not alter the current consistency model. Increasing the consistency level would incur higher costs without necessarily benefiting existing applications, while decreasing it could compromise the semantics of current file systems. NVLog adheres to the belief that the existing consistency level represents the best practice for current applications.
- P3: No performance downgrade to current file systems.** The goal of NVLog is to address the limitations of disk file systems rather than provide unbalanced performance like previous works [16, 26, 37, 40]. Therefore, NVLog should ensure that it does not degrade the performance of underlying file systems under any circumstances.
- P4: Lightweight design, minimal persistence footprint.** To ensure stability, the design of NVLog is kept as simple as possible. Additionally, NVLog occupies only the

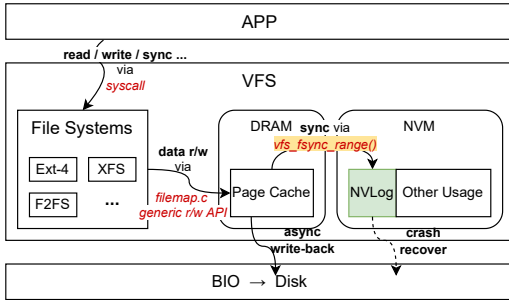


Figure 2: NVLog Architecture. Figure shows the position and data flow of NVLog inside the Linux kernel.

minimum necessary NVM space, allowing the remaining portion to be used for tiered caching or other optimization methods to further enhance system performance.

4 NVLog Design

NVLog is designed as a write-ahead log that exclusively absorbs sync writes, while all other normal read/write operations are still served by the DRAM page cache. Figure 2 illustrates the position and data flow of NVLog inside the Linux kernel. In this section, we first present the log structure design and the write process of NVLog. Second, we introduce sync semantic optimization to reduce write amplification in small, scattered sync writes. Then, we discuss the crash consistency challenges and solutions faced by NVLog. Finally, we cover the crash recovery and garbage collection processes.

4.1 Log Structure

As depicted in Figure 3, the fundamental design of NVLog consists of a series of logs on NVM. This log-based design offers two major benefits to NVLog: First, it eliminates the need for indexing, allowing for fast append-only writes (recall **I1**), which ensures high performance. Second, out-of-date data can be easily recycled, enabling NVLog to temporarily use only a small portion of the NVM space (recall **P4**). Both of these advantages have not been adopted in previous work [26,37], making NVLog a novel approach in leveraging NVM for file system acceleration.

4.1.1 Memory Arrangement

For each log series (inode log/super log, introduced in Section 4.1.2), the 64B log entries are first sequentially organized in a 4KB page. When the log exceeds the current page, another page is allocated. This new page is then linked to the previous page by a linked list, creating a chain of log pages. The traversal operation on the log is accelerated by prefetching.

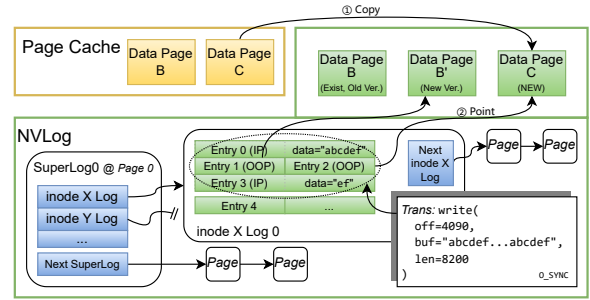


Figure 3: Log Structure. The detailed design of the data arrangement and the behavior of sync writes. The yellow parts are data pages in DRAM cache, the green parts are persistent data logs in NVM, and the blue parts are persistent metadata that helps NVLog to persist data.

4.1.2 Log Type

There are two types of logs in NVLog. The first type is the *super log*, which contains pointers that link to the log heads of all the inodes managed by NVLog. The second type of log is the *inode log*. Each file managed by NVLog has its own inode log, where all sync writes and metadata updates are recorded. For example, in Figure 3, the first entry in the super log, marked as inode X Log, maintains a pointer to the head log page of inode X, which is inode X Log 0. Subsequent log pages for inode X are linked together via a linked list. Each inode X log page contains plenty of synchronous update events (green squares in the figure) related to inode X.

There is only one global super log in NVLog, and its first log page (log head) is located at the physical 0 address of the NVM device. This placement ensures that NVLog can easily locate the super log directly after a power failure. The super log serves as the root of all logs, from which all persistent domain data can be found and replayed.

4.1.3 Log Entry

Each entry in the super log describes an inode log. The structure of the super log entry looks as follows:

```
struct superlog_entry
{
    dev_t          s_dev;
    unsigned long  i_ino;
    uint32_t       head_log_page;
    struct inode_log_entry *committed_log_tail;
}
```

The `s_dev` and `i_ino` fields are used to locate a specific file; the `head_log_page` pointer points to the first log page of this inode log; the `committed_log_tail` field indicates the current log tail of the inode. Whenever a new inode is delegated to NVLog, a new super log entry is created, pointing to the new inode log. Then when this file requires recovery,

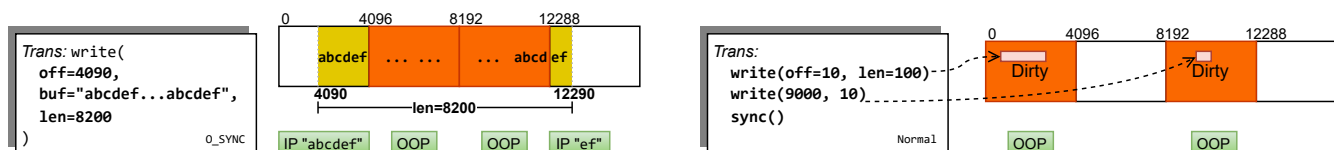


Figure 4: Sync Write Step. For `O_SYNC` writes, each write operation is divided into parts by page boundaries. Page-aligned (red) parts are recorded by OOP entries, and the unaligned (yellow) parts are recorded by IP entries. For normal writes followed by an `fsync`, all dirty pages are recorded by OOP entries.

NVLog retrieves its inode log by finding the corresponding super log entry. Note that the DRAM inode structure also holds a pointer to its NVLog log head, eliminating the need to search for its super log entry, so there’s no extra overhead during regular access.

The inode log captures all synchronous operations performed on the inode. Since NVLog manages NVM in pages, writes are naturally divided into segments no larger than a page. Consequently, if a write operation spans across $n - 1$ page boundaries, it may need to be recorded up to n times. Each entry in the inode log corresponds to a write segment, and the format of each entry is as follows:

```
struct inodelog_entry
{
    uint16_t    flag;
    uint64_t    file_offset;
    uint16_t    data_len;
    uint32_t    page_index;
    uint64_t    last_write;
    uint64_t    tid;
}
```

There are two types of inode log entries: out-of-place log entries (OOP entries) and in-place log entries (IP entries). In an OOP entry, the data is stored in a separate page pointed by the `page_index` field. In contrast, an IP entry stores the data within the log zone itself. The type of entry is distinguished by the `page_index` field: if `page_index` is 0, the entry is an IP entry; otherwise, it is an OOP entry.

NVLog uses the two types of entries to record data of different lengths. For whole-page data, NVLog employs OOP entries to perform large, page-aligned shadow-paging writes. On the one hand, this approach facilitates data page allocation and reclamation. On the other hand, as the new OOP data page is filled with fresh data, there’s no need to copy the old data in write process (e.g., Entry 1 and Data Page B’ in Figure 3). For smaller, unaligned write segments, which can have arbitrary sizes, NVLog uses IP entries to record them according to their actual length (e.g., Entry 0 and Entry 3 in Figure 3). By leveraging the byte-addressable nature of NVM, this method helps to avoid write amplification.

Except for the differences mentioned above, OOP entries and IP entries manipulate the remaining fields in the same way. The `file_offset` field indicates the position in the

file that the current entry is writing to. The `data_len` field specifies the length of the current write. The `last_write` field is reserved for locating the previous write at the same position during backtracking (see Section 4.6). The `tid` field marks a transaction. The `flag` field indicates the state of an entry.

Apart from the two types of write entries, there is also a metadata update entry to record changes to the inode’s metadata, and a write-back record entry, which will be discussed in Section 4.5. Their structures are straightforward and can be found in our open-source code, so we will not list them here.

4.2 System Modification

Let’s first examine the sync write steps of the Linux kernel. When a write is performed, the application first uses a `syscall` to invoke the file system. The file system then writes the data to the DRAM page cache and marks the relevant pages as dirty. Later, when a sync is performed, `vfs_fsync_range` is called to synchronously write the dirty pages back to the disk. If no sync is performed, the dirty pages will eventually be written back to the disk asynchronously in the background.

Previous works intercept the `syscall` by overlaying an NVM file system, thereby achieving write absorption on lower disk file systems. However, this approach separates the NVM accelerator from the system page cache, leading to double indexing overhead and inefficient redundant operation redirection. In contrast, NVLog absorbs sync writes within `vfs_fsync_range`. This allows NVLog to focus solely on absorbing sync writes while maintaining the benefits of the DRAM page cache provided by the system, resulting in higher performance (recall **P3**) and lower resource usage (recall **P4**).

It is important to note that NVLog does not alter the existing functions of the DRAM page cache. Writing data to NVLog does not clear the dirty flag of the page. NVLog simply converts synchronous disk write-backs into asynchronous ones, and uses NVM to perform immediate data persistence. This ensures that all data will eventually appear on the disk. Meanwhile, an extra flag is added to track the dirty pages that have been absorbed by NVLog. This ensures that the same write will not enter NVLog multiple times.

By converting sync writes into periodical async writes, NVLog also provides some additional benefits. For `fsync` op-

erations, each write to disk typically involves a data write and a metadata write. NVLog can first convert both of these writes into NVM writes and later, during an async write-back, aggregate the multiple metadata updates caused by the data updates, writing them to disk in one operation. This reduces the write pressure on the disk and extends its lifespan (especially for SSDs). For `fdatasync` operations, if it's an append write, new disk blocks also need to be allocated through metadata operations. Similarly, NVLog can optimize block allocation by aggregating multiple synchronous writes over a period of time to improve contiguous block allocation.

4.3 Sync Write Steps

Sync writes can be categorized into two types: `O_SYNC` writes and `fsync`-like calls (e.g., `fsync` and `fdatasync`). These two kinds of sync leads to different system behaviors, as shown in Figure 4. For normal writes, the behavior is straightforward for NVLog: record all dirty pages to NVM. To better illustrate the NVLog write process, we will take the more complex `O_SYNC` writes as an example.

To respect **P2**, each sync write operation is regarded as a transaction in NVLog, and is assigned an auto-increment id. The write transaction is broken into segments according to the page boundaries it crosses. For each segment, a log entry is appended to the end of the inode log of the current file. Aligned whole-page segments are recorded by OOP entries, with the data copied from the DRAM cache to a newly allocated NVM data page (e.g., ① in Figure 3). Unaligned segments are recorded by IP entries, with the data copied to the entry's data zone. Entry bodies are filled according to the definition in Section 4.1.3. Note that even if there is a previous OOP NVM data page found for the same offset (e.g., Data Page B in Figure 3), we cannot reuse it. Otherwise, it might result in the loss of data from the previous transaction if a crash occurs before the current transaction ends.

To further ensure **P2**, several techniques are applied. First, the `committed_log_tail` of the inode log is only updated atomically after all the segments in a transaction have finished, ensuring its integrity. Second, due to the presence of CPU caches, writes to NVM may return before they are eventually persisted to the NVM device. To eliminate this inconsistency risk, NVLog uses cache line write-back (e.g., Intel's `clwb`) to explicitly instruct the CPU to flush data back to NVM. If the system support `eADR` [2], the cache line write-back process can be omitted, allowing NVLog to achieve better performance. Third, memory barriers (`sfence`) are employed to maintain the ordering of store operations before and after consistency-critical points. In NVLog, only two barriers are used. The first is placed after all transaction segments are logged and before the `committed_log_tail` is updated, ensuring that the transaction is complete before it can be seen. The second barrier is placed after the commit and before the

start of the next transaction to maintain order between transactions.

Algorithm 1 Active sync mechanism.

```

1: global variables
2:   should_active_cnt           ▷ Counter before activate.
3:   should_deact_cnt          ▷ Counter before deactivate.
4: end global variables
5: procedure MARKSYNC(file, written_bytes,
                      dirty_pages, sensitivity)
6:   ▷ This is called on each sync.
7:   if written_bytes < dirty_pages * 4096 then
8:     should_active_cnt ← should_active_cnt + 1
9:     if should_active_cnt ≥ sensitivity then
10:      file.flags ← file.flags | O_SYNC
11:      should_deact_cnt ← 0
12:    end if
13:  end if
14: end procedure
15: procedure CLEARSYNC(file, written_bytes,
                      dirty_pages, sensitivity)
16:   ▷ This is called on each write.
17:   if written_bytes ≥ dirty_pages * 4096 then
18:     should_deact_cnt ← should_deact_cnt + 1
19:     if should_deact_cnt ≥ sensitivity then
20:      file.flags ← file.flags & ~O_SYNC
21:      should_active_cnt ← 0
22:    end if
23:  end if
24: end procedure

```

4.4 Active Sync Optimization

As depicted in Figure 4, `O_SYNC` and `fsync` exhibit different behaviors. `O_SYNC` is a flag indicating that a file or a mount point should always be written synchronously. Therefore, sync can be performed within the write syscall, at which point the exact range of data that needs to be recorded in NVLog is known. In contrast, the `fsync` operation is a post-write instruction, which means that at this point, we only know the dirtied data in the granularity of page instead of bytes. Consequently, if small, scattered writes occur before an `fsync`, the sync operation will write all the dirtied full pages to the NVM, resulting in severe write amplification. For example, in Figure 4, the pink fragments will cause all contents of the red dirty pages to be written to NVM.

To address the write amplification problem of `fsync`, we introduce *active sync*. The main idea is to predict whether subsequent synchronous writes will be more efficient if performed on full pages or at the byte level. Since application access patterns typically exhibit temporal locality, we can use past access patterns to predict future behavior. We track the count of dirtied pages and the number of written bytes be-

tween two sync operations, and then compare them according to Algorithm 1. Based on this comparison, we can dynamically apply or withdraw the `O_SYNC` flag on files that are not originally marked as `O_SYNC`.

Taking Figure 4 Normal as an example, the total written byte count is 110, and the dirtied page count is 2. In Normal mode, the sync operation results in 4096×2 bytes being written to NVLog. However, if it were in `O_SYNC` mode, only 110 bytes would need to be written. Based on this observation, we can predict that the performance of this file may be better in `O_SYNC` mode in the near future, so we proactively mark it as `O_SYNC`. Note that `written_bytes` may larger than `dirty_pages \times 4096` because there can be repeated writes to a single page.

The `sensitivity` parameter is used to tune the algorithm for different workloads to prevent thrashing. Our tests show that, unless the sync access pattern of the application is highly irregular, setting this value to 2 generally enhances the performance of small sync writes for most daily applications.

4.5 Consistency between NVM and Disk

We now have two separate write paths to NVM and disk. Synchronous writes are persisted directly to NVLog and may involve fewer bytes than a full page. In contrast, any write operation, whether synchronized or not, triggers asynchronous whole-page write-backs from DRAM to disk.

Pages on the underlying disk of NVLog always contain the correct data with the appropriate intra-page write sequences, because a disk page is written as a checkpoint of the page in DRAM cache, and the file system always ensures that the cached page is accurate. However, since we have converted all sync writes to asynchronous writes on disk, there is a risk that the disk page might persist an older version of the data if a power failure occurs before a synchronized dirty page is written back to the disk. Fortunately, NVLog ensures that a fresher version of the data is persisted on NVM. Nevertheless, because we only persist the necessary bytes for sync writes and not all bytes of all dirty pages, there is a potential for inconsistency issues, which could violate P2.

Figure 5 illustrates a typical scenario of inconsistency between the data on disk and NVM. At time t_2 , the page cache, disk, and NVM are all in a consistent state with version V1. By time t_7 , the disk holds the latest version V3, while NVM contains the previous version V2, which can be reconstructed from V1 and recorded O1. The discrepancy arises because O2 is not a sync write, and therefore, it is not persisted to NVM. The first issue is that if a crash occurs at t_7 , the recovery process will reconstruct V2 from NVM and overwrite the V3 on the disk, leading to an unexpected data rollback.

A more critical issue arises at t_{10} . At this point, a new sync write O3 has been performed and successfully recorded by NVM, but the new data V4 has not yet been written back to the disk. In the event of a crash here, we would only be

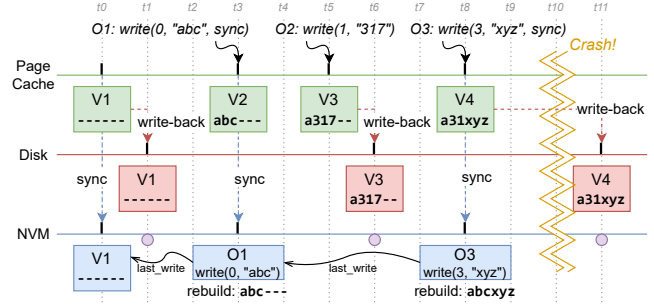


Figure 5: Page consistency on heterogeneous devices.

able to rebuild `abcxyz` from NVM, which does not reflect any expected version. The data has been messed up on NVM at this point, yet the disk only has the previous version V3 instead of the latest sync version V4.

The key to these frustrating problems is that we don't have the exact time sequence of disk writes and NVM writes. To address these problems, we persist disk write-back events on NVM (indicated by the purple bubbles on the NVM timeline in Figure 5) to maintain a global clock. Whenever a disk write-back occurs, if (and only if, for the sake of performance) a valid previous entry exists, a write-back entry is appended to the inode log, marking the previous writes on this page as expired. During recovery, only unexpired operations are replayed to the page version on the disk. For example, at t_{10} , we have O1 and O3 on the NVM, and V3 on the disk. The write-back entry (purple bubble) between O1 and O3 indicates that O1 and its predecessors are expired. So we only replay O3 to V3, resulting in `a31xyz`, which is the correct version reflecting the lost V4. This mechanism ensures that NVLog will always rebuild the latest data without rollback or errors.

It is worth mentioning that this algorithm is a key factor in enabling NVLog's performance to outperform previous works (recall I2 and P3). Without this algorithm, absorbing only sync writes to NVM while maintaining the DRAM-disk path for other normal writes would result in confusion between NVM and disk data versions. To avoid this confusion, for example, P2CACHE absorbs all writes to NVM, thus leading to unnecessary performance degradation under asynchronous writes.

4.6 Crash Recovery

To recover the data on NVLog to the disk after a power failure, a crash replay procedure is introduced. This procedure involves multiple passes through each inode log to recover each file. First, it traverses the entire inode log. During this pass, log entries associated with the same data page offset are linked together in sequence via the `last_write` field of each entry, and the latest write entry for each data page is saved into an index. After the whole index is built, the replay starts.

For each page, the rebuilder walks from its latest entry back to the earliest via the `last_write` field. Once a write-back entry or an OOP entry is found, the walk halts because the previous data are either expired or overwritten. The data from the traversed entries are then replayed to the data page on the disk.

Note that our scanning ends at `committed_log_tail`, and any uncommitted partial writes after this point will be dropped. This ensures that even though a single write might span multiple pages, it will be recovered in an all-or-nothing manner. Also note that NVLog does not conflict with the journaling mechanism (e.g., JBD) of the file system, as NVLog only records events, not data blocks. After a crash, as long as the underlying file system remains intact, NVLog enables the file system to catch up to the expected status. Therefore, in such cases, running `fsck` should be the first step, followed by NVLog recovery. Since the recovery process only occurs after a crash reboot, it does not affect the performance of regular operations. We conducted various crash experiments on the experimental platform described in Section 6, and the results show that the recovery time is usually around 10 seconds.

As we can see, NVLog postpones the index-building process to the recovery period, whereas previous works maintain their index during regular runtime. The elimination of runtime indexing is another factor contributing to our lightweight and efficient design (recall **I1**).

4.7 Garbage Collection

To reclaim NVM space from NVLog and respond to **P4**, a garbage collector is provided as a kernel thread. This garbage collector periodically runs in the background, walking through the log pages to check for any log or data pages that are no longer needed. A log entry becomes obsolete when it expires due to a subsequent write-back or is overwritten by a later OOP entry. A data page is considered useless if its associated log entry is obsolete. A useless data page is reclaimed as soon as it is identified, and a log page is reclaimed once all its entries are deemed useless. The walk stops before the latest log page of each inode, as the latest page is obviously still in use. This scanning process does not require any locks, ensuring that it does not interfere with foreground operations.

Note that emerging NVM technologies, such as RRAM and MRAM, may have smaller capacities compared to PCM, which could lead to NVLog encountering situations where the NVM is full. In such cases, NVLog will fall back to performing sync writes to disk and wait for the GC to free up available NVM pages. NVLog does not need to wait for GC to complete; as long as there are free pages, it can resume using the NVM sync path.

5 Implementation

NVLog is implemented in the Linux kernel 5.15 (LTS) with 7.3K lines of kernel code and no more than 1K lines of code for auxiliary tools. The changes to the kernel code are limited to the Virtual File System (VFS) (0.3K LOC), the memory management system (6.2K LOC), and the drivers (0.8K LOC).

The VFS code is minimally modified to transfer necessary events to NVLog without altering the current API semantics, thus maintaining full compatibility and transparency for existing applications and file systems (recall **P1**). The memory management system (MM) contains the majority of our work. All functions of NVLog discussed in Section 4 are implemented in MM. Meanwhile, extra page flags are added to track page status, and the mark/clear-dirty and write-back functions are modified to handle the new flags. A new driver module is added to initialize DAX NVM devices and configure NVLog. Furthermore, some utilities are provided to initialize, configure, and monitor NVLog from user space. The prototype is open source and can be found at <https://github.com/BugJLU/NVLog>.

6 Evaluation

In this section, we evaluate NVLog using micro- and macro-benchmarks, to demonstrate the performance advantages and limitations against other file systems. We compare NVLog with NOVA and SPFS, which respectively represent the state-of-the-art work in NVM file systems and NVM overlay accelerators. We did not select P2CACHE as the representative because its open-source code is incomplete to run properly. Unless specified, file systems and workloads use default configurations (e.g., ordered-journaling for Ext4).

All the experiments are conducted on a system equipped with Intel Xeon 5218R (20 cores), 128GB DRAM, 256GB Intel Optane PMEM (128GB x2, interleaved), Samsung PM9A3 1.92TB NVMe SSD, and Ubuntu 20.04. Note that the SSD used in the evaluation has a high speed, whereas the bandwidth of the NVM is limited because only two modules are installed. Therefore, our experiments roughly represent the lower bound of acceleration that NVLog can bring. In systems with slower storage (e.g., SATA SSDs or HDDs) and higher bandwidth NVM (e.g., more PMEM modules installed), the performance improvement ratio of NVLog will be much higher than the results reported in this section.

6.1 Microbenchmarks

We use multiple microbenchmarks to evaluate the performance of NVLog and other file systems. In this section, we choose both Ext-4 and XFS as baselines to demonstrate the flexible adaptability of NVLog. Similar to previous work, NVLog does not accelerate cold I/O from disk, as most workloads benefit from caching. Therefore, to better demonstrate

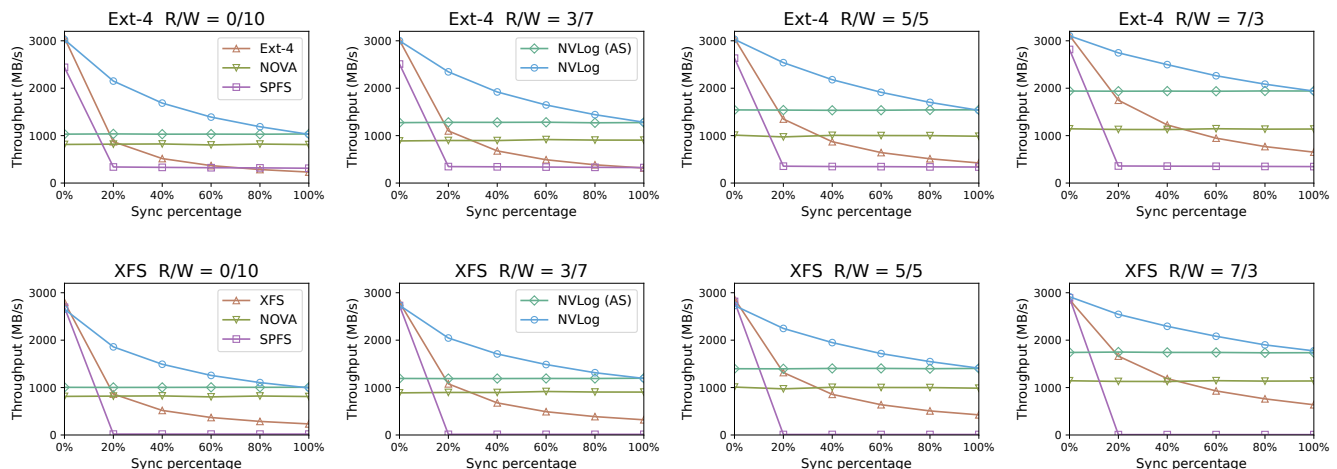


Figure 6: Read, write, and sync mixed tests under 4KB random access. AS: all writes are forced to be synchronized.

the advantages of NVLog’s design in common use cases, our experiments are conducted with data pre-loaded into the cache (by reading the test file for one pass).

6.1.1 Mixed Operation Access

To comprehensively illustrate the performance advantages of NVLog’s design, we first deploy 4KB random read/write tests with varying r/w ratios (0/10, 3/7, 5/5, 7/3) and different sync write percentages (from 0% to 100% in steps of 20%). NVLog can absorb sync operations on demand due to its consistency design described in Section 4.5. To highlight the impact of this design, we also test NVLog (AS, always-sync) in this experiment, representing performance without the aid of the consistency design, where all writes must be persisted to NVM to ensure consistency, similar to the strategy used by P2CACHE.

The experimental results are shown in Figure 6. Thanks to our DRAM-NVM cooperative design, NVLog outperforms NVM FS, disk FS, and NVM-based FS accelerators in most cases. In non-sync workloads, by leveraging the DRAM page cache, NVLog performs similarly to its baseline disk FS, achieving speeds up to 3.72x, 2.93x, and 1.24x faster than NOVA, NVLog (AS), and SPFS, respectively. In partial-sync workloads, NVLog outperforms the disk FS, NOVA, and SPFS by up to 4.44x, 2.62x, and 324.11x, respectively. In this test, SPFS demonstrates poor performance. Our performance breakdown shows that 97% of its time is spent on indexing, indicating that its indexing mechanism suffers from significant degradation in random access scenarios. The results show that NVLog consistently maintains a good balance between DRAM and NVM access across various sync levels. Additionally, it is evident that NVLog is the only solution that does not introduce any slowdown to the legacy disk FS.

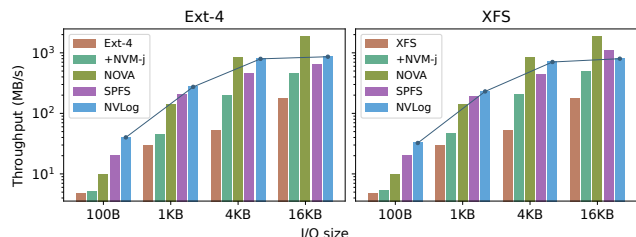


Figure 7: Sync performance under different I/O sizes. +NVM-j refers to the performance of placing the baseline file system’s journal on NVM.

6.1.2 Pure Sync with Different I/O Sizes

To demonstrate NVLog’s performance characteristics under pure sync scenarios, we evaluated different systems with sequential sync writes across various I/O sizes. The results are shown in Figure 7. In these tests, NVLog consistently accelerates its underlying disk FS across all sizes, achieving up to a 15.09x acceleration with Ext-4 and 13.54x with XFS. We also evaluated the method of placing file system journal on NVM, marked as “+NVM-j” in the figure. Compared to the NVM-journaling method, NVLog can still provide up to 7.73x acceleration because (1) NVLog can bypass both the FS journaling and data write phases, whereas NVM-journaling only accelerates the journaling phase, and (2) NVLog is optimized for small-granularity writes and has a shorter call stack.

For small writes, NVLog can even outperform NOVA by up to 4.13x, thanks to its arbitrary-length log design that eliminates write amplification. However, NVLog fails to compete with NOVA (and SPFS in XFS) for large 16KB synchronous writes. The reasons are twofold: (1) NVLog cannot eliminate the software overhead of the underlying FS and page cache,

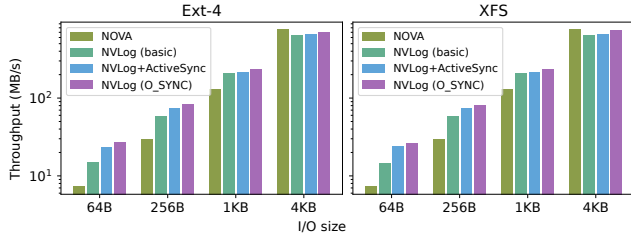


Figure 8: Active sync optimization.

and (2) NVLog writes to both DRAM and NVM, while NOVA and SPFS (after successful prediction) write only to NVM. The high performance of NOVA and SPFS in this scenario results from slowdowns in other scenarios, such as reads, as confirmed in our previous experiments. In line with our principles and goals, particularly **P1** and **P3**, we believe that NVLog’s moderate, non-aggressive acceleration is sufficient to enhance lower disk FS while providing better performance across a broad range of mixed daily scenarios.

6.1.3 Active Sync

To investigate the performance benefits of the active sync mechanism, we test NVLog and NOVA with small sync writes of less than 4KB. In this test, synchronization is enforced by issuing `fsync` calls after each write operation. We also measured the performance of the `O_SYNC` version NVLog as an upper bound. Figure 8 shows the result. With active sync enabled, NVLog’s performance is up to 1.62x faster than the vanilla one, and up to 3.22x faster than NOVA under 64B small writes. The smaller the I/O sizes, the more pronounced the effect of active sync, as it can reduce more write amplification. The active sync mechanism helps `fsync` achieve 86.21% to 94.17% of the performance of `O_SYNC`, demonstrating the effectiveness of this optimization.

6.1.4 Scalability

To measure the scalability of NVLog, we conduct a 4KB random read/write test with multiple threads accessing different files, varying the number of threads from 1 to 16. The read-write ratio is set to 1:1, with all writes being synchronized. The result is shown in Figure 9. As the number of threads increases, NVLog scales well and outperforms all competitors. For Ext-4-based systems, NVLog outperforms NOVA, Ext-4, and SPFS by up to 1.94x, 3.11x, and 8.87x, respectively. For XFS-based systems, NVLog outperforms NOVA, XFS, and SPFS by up to 1.91x, 2.93x, and 28.18x, respectively. NVLog performs well because (1) it effectively uses DRAM and NVM separately to serve reads and writes, and (2) it does not introduce any additional locks to the software stack. SPFS shows poor performance due to its secondary index that

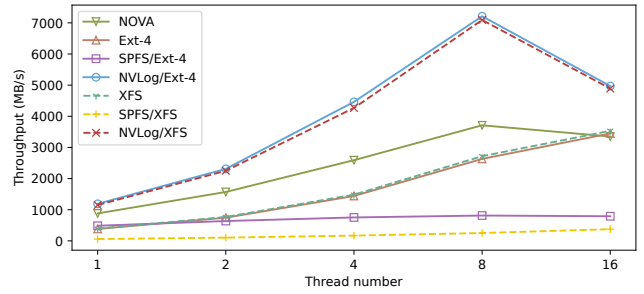


Figure 9: Scalability under random r/w test.

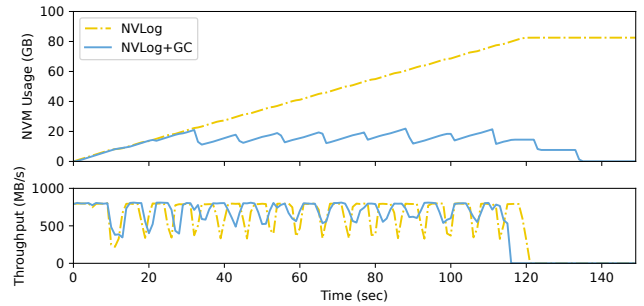


Figure 10: Garbage collection performance. The figure shows the NVM usage and the throughput of NVLog with or without garbage collection.

fails to scale over multiple threads, and its read-after-sync slowdown.

Note that there is a performance degradation from 8 to 16 threads in both NOVA and NVLog. This is due to the saturation of the limited NVM write bandwidth, as our testbed only has two interleaved PM modules. The performance drop in NVLog is not related to our design, since NVLog does not introduce any additional locks. The test is stopped at 16 threads because our testbed only has 20 cores.

6.1.5 Garbage Collection

We perform an 80GB sync write test on NVLog and track both the NVM usage and throughput to demonstrate the effectiveness and performance impact of garbage collection. The results are shown in Figure 10. The garbage collection scan interval is set to 10 seconds, which is why the figure shows a drop in NVM usage every 10 seconds. In the first two rounds, the space reclamation is not obvious because the write-back process has not yet been activated. Throughout the entire process, the NVM usage of the garbage collection-enabled NVLog does not exceed 22GB and eventually drops to nearly zero. The performance fluctuations seen in the throughput are caused by the page allocator. Our implementation includes a per-CPU NVM page pool that helps improve performance. The observed performance drops indicate that the per-CPU pool is drained and is allocating new pages. Garbage collec-

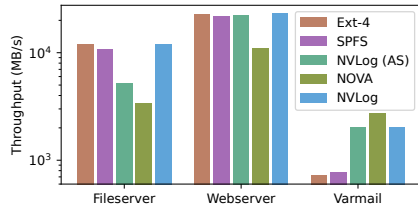


Figure 11: Filebench performance.

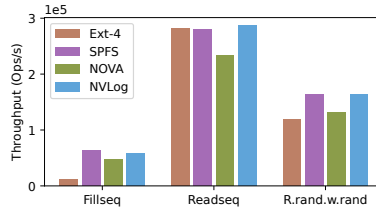


Figure 12: RocksDB performance.

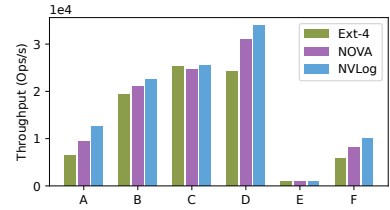


Figure 13: YCSB benchmark on SQLite.

tion alleviates this situation by returning unused pages back to the per-CPU pool. Overall, the garbage collector is effective and does not negatively impact performance.

6.1.6 Capacity Limit

We recognize that emerging NVM technologies may have relatively small capacities, so we test the performance of NVLog under capacity-limited conditions. We restrict the available NVM size to 10GB (about half of the peak usage reported in Figure 10) and then use `db_bench` (will be introduced in Section 6.2.2) with its typical workloads to test NVLog’s performance. For read (`readseq`) and uniformly random read-write (`readrandomwriterandom`) workloads, limiting the NVM capacity has no impact on performance. For the fully synchronous write (`fillseq`) workload, performance drops by 57% compared to the unlimited case shown in Figure 12, but it is still 2.25x faster than Ext-4.

6.2 Macrobenchmarks

Due to the similar performance trends observed across different disk FS baselines in the microbenchmarks, we will use only Ext-4 as the baseline in the macrobenchmarks.

6.2.1 Filebench

Filebench [3] provides 3 representative macrobenchmark scripts to simulate different server workloads: `fileserver`, a non-sync, write-intensive workload with a 1:2 r/w ratio; `webserver`, a read-intensive workload with a 10:1 r/w ratio; and `varmail`, a balanced read/sync-write 1:1 workload with small I/O size. The detailed configurations of these workloads are listed in Table 1.

Table 1: Filebench workload configurations.

Workload	File size (avg)	I/O size (r/w)	Threads	R/W ratio	# of files
Fileserver	128KB	1MB/16KB	16	1:2	10000
Webserver	64KB	1MB/16KB	16	10:1	1000
Varmail	16KB	1MB/16KB	16	1:1	10000

The result of the test is shown in Figure 11. In the `fileserver` and `webserver` workloads, NVLog, SPFS, and

Ext-4 exhibit similar performance, significantly outperforming NOVA due to their use of a fast DRAM page cache. For example, NVLog demonstrates 3.55x and 2.10x the performance of NOVA in the `fileserver` and `webserver` workloads, respectively. NVLog (AS) requires all writes to be synchronized, which results in worse performance than standard NVLog in `fileserver`. Nonetheless, NVLog (AS) still outperforms NOVA by 1.53x, primarily due to the DRAM-enabled reads. In the `varmail` workload, NVLog is 2.84x and 2.65x faster than Ext-4 and SPFS, respectively, but 25.98% slower than NOVA. SPFS fails to effectively accelerate Ext-4 in `varmail` because it requires a prediction period before absorbing sync writes to NVM. However, `varmail` synchronously writes to scattered files only twice per file, preventing SPFS from predicting and absorbing most of these scattered sync writes. The relatively lower performance of NVLog compared to NOVA in `varmail` is due to the double-write to both DRAM and NVM. Nevertheless, we believe that retaining the DRAM cache helps to provide a more balanced performance in daily workloads, where sync operations constitute only a small portion.

6.2.2 RocksDB

RocksDB [9] is an LSM-tree-based key-value database for server workloads. Data written to RocksDB is first recorded in a write-ahead log (WAL) and then asynchronously written to the LSM tree (SST files). Reading data from RocksDB involves reading from the SST files. RocksDB includes `db_bench` as its testing suite. To demonstrate NVLog’s performance under different conditions, we selected three tests in `db_bench`: sequential write (`fillseq`), sequential read (`readseq`), and multi-threaded mixed read/write (`readrandomwriterandom`). The value size is set to 4KB, and the level 1 file size is set to 512MB. We initially run a `fillseq` to create the database, enable sync mode for each test, and remove the database after each test. Figure 12 shows the performance of RocksDB.

For `fillseq`, SPFS, NOVA, and NVLog are 5.83x, 4.33x, and 5.23x faster than Ext-4, respectively. The low performance of Ext-4 is due to its WAL sync writes suffering from the low speed of the disk. NOVA performs slower than SPFS and NVLog because of its write amplification for small meta-data writes, which is a result of its copy-on-write design.

For `readseq`, Ext-4 and NVLog perform similarly, and both outperform NOVA. This is because, in NVLog and Ext-4, read operations are served by the fast-path DRAM, whereas NOVA can only perform reads on NVM. SPFS also provides comparable high speed, as it avoids its inherent read-after-write slowdown. Specifically, RocksDB reads from SST files, which are previously written to disk in large chunks (tens or hundreds of MB) synchronously. SPFS does not absorb sync writes larger than 4MB, allowing it to continue serving RocksDB reads from DRAM rather than NVM.

For `readrandomwriterandom`, NVLog performs 1.38x faster than Ext-4 and 1.24x faster than NOVA. The advantage of NVLog stems from its cooperative DRAM-NVM design. SPFS achieves similar performance to NVLog, once again due to its ability to skip large bulk syncs.

Overall, NVLog and SPFS both achieve balanced performance on RocksDB, with higher speeds than Ext-4 for writes and comparable speeds for reads, while NOVA provides reasonable write speeds but deteriorated read speeds. However, note that SPFS encountered several crashes when attempting to clean up the existing database and failed to run under RocksDB's `O_DIRECT` mode. By contrast, NVLog and NOVA both demonstrated good robustness during the test.

6.2.3 SQLite

SQLite [10] is a lightweight embedded SQL database engine widely used across PC and mobile applications. To evaluate whether SQLite can benefit from NVLog, we use YCSB [13, 25], a common benchmark for database performance. YCSB includes six different workloads, labeled A through F, to cover a variety of application scenarios. In this test, SQLite is in FULL synchronous mode, and the record size is 4KB. The user space cache is set to 0 to fully demonstrate the performance of the underlying software stack. The result of SQLite's performance is illustrated in Figure 13.

In workloads A, B, D, and F, which involve writing to the database, NVLog outperforms both Ext4 and NOVA. NVLog accelerates Ext-4 by up to 1.91x due to the high persistence speed of NVM. NVLog also surpasses NOVA by up to 1.33x because its arbitrary-length log structure and active sync mechanism efficiently handle small metadata updates. In workloads C and E, which are read-only, the performance differences among the tested systems are minimal. Since we have eliminated the impact of user cache, this is likely because the query execution time dominates, making the performance differences in page cache and storage media less significant. SPFS did not appear in this experiment due to recurring crashes during testing.

7 Conclusion

In this paper, we propose NVLog, an NVM-based write-ahead log designed to transparently accelerate traditional disk file

systems. NVLog is the only approach that effectively balances performance across DRAM, NVM, and disk, ensuring that the enhancement does not introduce any slowdowns to the existing storage stack. Our experiments demonstrate that NVLog outperforms previous solutions in most common use cases, while also maintaining a stable and lightweight characteristic.

Acknowledgments

We would like to thank our shepherd, Peter Macko, for his detailed suggestions and guidance during the revision phase of our paper. We also wish to express our gratitude to Professor Guangyan Zhang from Tsinghua University and Professor Cheng Li from University of Science and Technology of China for their guidance and assistance. The corresponding author of this paper is Juncheng Hu.

This work is supported by the National Key R&D Program under Grant No. 2024YFB3310200, and by the Key Scientific and Technological R&D Plan of Jilin Province of China under Grant No. 20230201066GX, and by the Central University Basic Scientific Research Fund Grant No. 2023-JCXX-04.

Availability

The prototype of NVLog is implemented and available at <https://github.com/BugJLU/NVLog>.

References

- [1] DAX. <https://www.kernel.org/doc/Documentation/filesystems/dax.txt>.
- [2] eADR: New Opportunities for Persistent Memory Applications. <https://www.intel.com/content/www/us/en/developer/articles/technical/eadr-new-opportunities-for-persistent-memory-applications.html>.
- [3] Filebench. <https://github.com/filebench/filebench>.
- [4] Intel® Optane™ Persistent Memory. <https://www.intel.com/content/www/us/en/products/docs/memory-storage/optane-persistent-memory/overview.html>.
- [5] JEDEC. <https://www.jedec.org/>.
- [6] Journal (jbd2) — The Linux Kernel documentation. <https://www.kernel.org/doc/html/latest/filesystems/ext4/journal.html#external-journal>.
- [7] Migrate Pages in lieu of discard. <https://lwn.net/Articles/860215/>.

- [8] MS-SSD – Samsung – Memory Solutions Lab. <https://samsungmsl.com/cmmh/>.
- [9] RocksDB. <http://rocksdb.org/>.
- [10] SQLite Home Page. <https://www.sqlite.org/>.
- [11] Compute Express Link. <https://computeexpresslink.org/>, September 2023.
- [12] Youmin Chen, Youyou Lu, Bohong Zhu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Jiwoo Shu. Scalable Persistent Memory File System with Kernel-Userspace Collaboration. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 81–95, 2021.
- [13] Brian Cooper. Brianfrankcooper/YCSB. <https://github.com/brianfrankcooper/YCSB>, September 2024.
- [14] Lixiao Cui, Kewen He, Yusen Li, Peng Li, Jiachen Zhang, Gang Wang, and Xiaoguang Liu. SwapKV: A Hotness Aware In-Memory Key-Value Store for Hybrid Memory Systems. *IEEE Transactions on Knowledge and Data Engineering*, 35(1):917–930, January 2023.
- [15] Mingkai Dong, Heng Bu, Jifei Yi, Benchao Dong, and Haibo Chen. Performance and protection in the ZoFS user-space NVM file system. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 478–493, Huntsville Ontario Canada, October 2019. ACM.
- [16] Subramanya R. Dulloor, Sanjay Kumar, Anil Keshavamurthy, Philip Lantz, Dheeraj Reddy, Rajesh Sankaran, and Jeff Jackson. System software for persistent memory. In *Proceedings of the Ninth European Conference on Computer Systems - EuroSys '14*, pages 1–15, Amsterdam, The Netherlands, 2014. ACM Press.
- [17] Zhiyuan Guo, Zijian He, and Yiyang Zhang. Mira: A Program-Behavior-Guided Far Memory System. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, pages 692–708, New York, NY, USA, October 2023. Association for Computing Machinery.
- [18] M. Hosomi, H. Yamagishi, T. Yamamoto, K. Bessho, Y. Higo, K. Yamane, H. Yamada, M. Shoji, H. Hachino, C. Fukumoto, H. Nagao, and H. Kano. A novel non-volatile memory with spin torque transfer magnetization switching: Spin-ram. In *IEEE International Electron Devices Meeting, 2005. IEDM Technical Digest.*, pages 459–462, Tempe, Arizon, USA, 2005. IEEE.
- [19] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amirsaman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dulloor, Jishen Zhao, and Steven Swanson. Basic Performance Measurements of the Intel Optane DC Persistent Memory Module, August 2019.
- [20] Zhicheng Ji, Kang Chen, Leping Wang, Mingxing Zhang, and Yongwei Wu. Falcon: Fast OLTP Engine for Persistent Cache and Non-Volatile Memory. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, pages 531–544, New York, NY, USA, October 2023. Association for Computing Machinery.
- [21] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. Strata: A Cross Media File System. In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, pages 460–477, New York, NY, USA, October 2017. Association for Computing Machinery.
- [22] Benjamin C. Lee, Engin Ipek, Onur Mutlu, and Doug Burger. Architecting phase change memory as a scalable dram alternative. In *Proceedings of the 36th Annual International Symposium on Computer Architecture, ISCA '09*, pages 2–13, New York, NY, USA, June 2009. Association for Computing Machinery.
- [23] Eunji Lee, Hyokyung Bahn, and Sam H. Noh. Unioning of the Buffer Cache and Journaling Layers with Non-volatile Memory. In *11th USENIX Conference on File and Storage Technologies (FAST 13)*, pages 73–80, 2013.
- [24] Geonhee Lee, Hyeon Gyu Lee, Juwon Lee, Bryan S. Kim, and Sang Lyul Min. An Empirical Study on NVM-based Block I/O Caches. In *Proceedings of the 9th Asia-Pacific Workshop on Systems, APSys '18*, pages 1–8, New York, NY, USA, August 2018. Association for Computing Machinery.
- [25] Youngjae Lee. Ls4154/YCSB-cpp. <https://github.com/ls4154/YCSB-cpp>, September 2024.
- [26] Zhen Lin, Lingfeng Xiang, Jia Rao, and Hui Lu. P2CACHE: Exploring Tiered Memory for In-Kernel File Systems Caching. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 801–815, 2023.
- [27] Yubo Liu, Yuxin Ren, Mingrui Liu, Hongbo Li, Hanjun Guo, Xie Miao, Xinwei Hu, and Haibo Chen. Optimizing File Systems on Heterogeneous Memory by Integrating DRAM Cache with Virtual Memory Management. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, pages 71–87, 2024.

- [28] Teng Ma, Mingxing Zhang, Kang Chen, Zhuo Song, Yongwei Wu, and Xuehai Qian. AsymNVM: An Efficient Framework for Implementing Persistent Data Structures on Asymmetric NVM Architecture. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, pages 757–773, New York, NY, USA, March 2020. Association for Computing Machinery.
- [29] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, pages 742–755, New York, NY, USA, March 2023. Association for Computing Machinery.
- [30] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. HeMem: Scalable Tiered Memory Management for Big Data Applications and Real NVM. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 392–407, Virtual Event Germany, October 2021. ACM.
- [31] Jie Ren, Dong Xu, Junhee Ryu, Kwangsik Shin, Daewoo Kim, and Dong Li. MTM: Rethinking Memory Profiling and Migration for Multi-Tiered Large Memory. In *Proceedings of the Nineteenth European Conference on Computer Systems*, EuroSys '24, pages 803–817, New York, NY, USA, April 2024. Association for Computing Machinery.
- [32] Chaoyi Ruan, Yingqiang Zhang, Chao Bi, Xiaosong Ma, Hao Chen, Feifei Li, Xinjun Yang, Cheng Li, Ashraf Aboulnaga, and Yinlong Xu. Persistent Memory Disaggregation for Cloud-Native Relational Databases. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, pages 498–512, New York, NY, USA, March 2023. Association for Computing Machinery.
- [33] Haris Volos, Sanketh Nalli, Sankarlingam Panneerselvam, Venkatanathan Varadarajan, Prashant Saxena, and Michael M. Swift. Aerie: flexible file-system interfaces to storage-class memory. In *Proceedings of the Ninth European Conference on Computer Systems*, EuroSys '14, pages 1–14, New York, NY, USA, April 2014. Association for Computing Machinery.
- [34] Jing Wang, Youyou Lu, Qing Wang, Minhui Xie, Keji Huang, and Jiwu Shu. Pacman: An Efficient Compaction Approach for Log-Structured Key-Value Store on Persistent Memory. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 773–788, 2022.
- [35] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. TMO: transparent memory offloading in datacenters. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, pages 609–621, New York, NY, USA, February 2022. Association for Computing Machinery.
- [36] H.-S. Philip Wong, Heng-Yuan Lee, Shimeng Yu, Yu-Sheng Chen, Yi Wu, Pang-Shiu Chen, Byoungil Lee, Frederick T. Chen, and Ming-Jinn Tsai. Metal–Oxide RRAM. *Proceedings of the IEEE*, 100(6):1951–1970, June 2012.
- [37] Hobin Woo, Daegyu Han, Seungjoon Ha, Sam H. Noh, and Beomseok Nam. On Stacking a Persistent Memory File System on Legacy File Systems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 281–296, 2023.
- [38] Chengwen Wu, Guangyan Zhang, and Keqin Li. Rethinking Computer Architectures and Software Systems for Phase-Change Memory. *ACM Journal on Emerging Technologies in Computing Systems*, 12(4):33:1–33:40, May 2016.
- [39] Lingfeng Xiang, Xingsheng Zhao, Jia Rao, Song Jiang, and Hong Jiang. Characterizing the performance of intel optane persistent memory: a close look at its on-DIMM buffering. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 488–505, Rennes France, March 2022. ACM.
- [40] Jian Xu and Steven Swanson. NOVA: a log-structured file system for hybrid volatile/non-volatile main memories. In *Proceedings of the 14th Usenix Conference on File and Storage Technologies*, FAST'16, pages 323–338, USA, February 2016. USENIX Association.
- [41] Guangyan Zhang, Dan Feng, Keqin Li, Zili Shao, Nong Xiao, Jin Xiong, and Weimin Zheng. Design and application of new storage systems. *Frontiers of Information Technology & Electronic Engineering*, 24(5):633–636, May 2023.
- [42] Xiaoyi Zhang, Dan Feng, Yu Hua, and Jianxi Chen. A Cost-Efficient NVM-Based Journaling Scheme for File Systems. In *2017 IEEE International Conference on Computer Design (ICCD)*, pages 57–64, November 2017.

- [43] Shengan Zheng, Morteza Hoseinzadeh, and Steven Swanson. Ziggurat: A Tiered File System for Non-Volatile Main Memories and Disks. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 207–219, 2019.
- [44] Shawn Zhong, Chenhao Ye, Guanzhou Hu, Suyan Qu, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, and Michael Swift. MadFS: Per-File Virtualization for Userspace Persistent Memory Filesystems. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 265–280, 2023.

A Artifact Appendix

Abstract

The artifact provides the source code of our NVLog prototype. It includes a modified Linux kernel that implements NVLog, and a series of utilities for configuration and evaluation.

Scope

This artifact includes a Linux kernel with the implementation of designs described in the paper, along with relevant scripts and documentation. It supports performance evaluation and facilitates a deeper understanding of the paper’s design details.

Contents

The NVLog artifact consists of three main components:

1. **Modified Linux Kernel (/linux-5.15.125):** This is the core of the artifact, containing the implementation of NVLog’s design.
2. **Utility Scripts:** These include scripts for setting up the environment and compiling the kernel (/dev_vm), as well as scripts for configuring NVLog in user space (/utils).
3. **Documentation:** This includes /README.md and additional README files linked within.

Additionally, the artifact provides instructions and scripts for performance evaluation of NVLog in /ae, supporting artifact evaluation and future work.

Hosting

The artifact is available on GitHub: <https://github.com/BugJLU/NVLog>.

Requirements

To run this prototype and reproduce the experimental results in this paper, Intel Optane PMEM is required.

Evaluation

Due to the large amount of experimental data presented in the paper and the slow performance of standard Ext-4 in synchronization test cases, reproducing all experiments during artifact evaluation could incur significant time cost. To address this, we have streamlined the experiments into three main claims and provided the necessary experimental scripts to validate these claims. The following are the three claims:

C1: NVLog can leverage NVM and DRAM to separately handle synchronous writes and other requests, achieving optimal performance under complex mixed read, asynchronous write, and synchronous write workloads. To validate this claim, we designed performance tests with different read/write ratios (R/W=0/10, 3/7, 5/5, 7/3), where 50% of the writes are set as synchronous. NVLog is expected to outperform NOVA, SPFS, and Ext4 in these tests.

C2: When performing sub-page granularity synchronous writes, NVLog can achieve optimal performance by fully utilizing the byte-granularity characteristics of NVM. To validate this claim, we conducted synchronous write performance tests at a 64B granularity. NVLog is expected to outperform NOVA, SPFS, and Ext4 in these tests.

C3: Due to NVLog’s design, which supports garbage collection, it temporarily occupies only a small amount of NVM. To validate this claim, we conducted an 80GB synchronous write test on NVLog. During most of the process, the NVM usage should be less than the write volume, and after garbage collection is completed, the NVM usage should be less than 1% of the total write volume.